



CS4051NI/CC4059NI Fundamentals of Computing

70% Individual Coursework

2024/25 Spring

Student Name: Wasiurrahman Musalman

London Met ID: 24046552

College ID: NP01NT4A240050

Assignment Due Date: Wednesday, May 14, 2025

Assignment Submission Date: Wednesday, May 14, 2025

Word Count: 3530

Project File Links:

Onedrive Drive Link:	Keep Google Drive URL of your Project Here with Anyone in Organization can View Option Enabled
-----------------------------	--

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

Turnitin Similarity Report:



8% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **27 Not Cited or Quoted 8%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **3 Missing Citation 1%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1%  Internet sources
- 0%  Publications
- 8%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Table of Contents

1. INTRODUCTION:.....	1
1.1 Aims:.....	2
1.2 Objectives:	2
1.3 Technologies used:.....	3
2. Discuss and Analysis:	6
2.1 Algorithm:	6
2.2 Flowchart:	7
2.3 Pseudocode:.....	8
2.4 Data Structure:.....	16
3. Program:	18
4. Testing:	24
4.1 Test 1: Showing implementation of try and except:	24
4.2 Test 2: Selection purchase and sale products	25
4.3 Test 3: File generation of purchase of product(s)	26
4.4 Test 4: File generation of sales process of product(s).	28
4.5 Test 5: Showing the update in stock of product(s)	30
5. Conclusion:	32
Bibliography	34
6. Appendix:	35

List of Figures

Figure 1 python.	3
Figure 2. IDLE Python.	4
Figure 3 draw.io	4
Figure 4. Microsoft Word.	5
Figure 5.Flowchart diagram of the whole program.	7
Figure 6 screen shot of the list example.	16
Figure 7 screen shot of the dictionary eg.	17
Figure 8 screen shot for the string eg.	17
Figure 9 screen shot for the tuple eg.	17
Figure 10 screen shot for the integer eg.	18
Figure 11 voucher for buy.	19
Figure 12 screen shot of buy invoice txt file.	20
Figure 13 complete output for the buy.	20
Figure 14 screen shot of invoice for sell.	21
Figure 15 screen shot for the sell txt.	21
Figure 16 complete output for sell.	22
Figure 17 exit program.	23
Figure 18 screen shot of the output after entering non numeric value.	24
Figure 19 Screenshot of providing negative input.	25
Figure 20 screen shot of providing non-existed value.	25
Figure 21 complete purchase process.	26
Figure 22 output in the sell	27
Figure 23 purchased products details in a text file.	27
Figure 24 output of entire process during sell.	28
Figure 25 output of invoice generated in shell during sell.	29
Figure 26 output of invoice generated in txt.	29
Figure 27 stock update after purchasing.	30
Figure 28 screen shot of update stock in txt file of buy product.	31
Figure 29 Stock updated after sell.	31
Figure 30 Screenshot stock updated in txt file after sell.	32

List of Tables

Table 1 Test to show the implementation of try except	24
Table 2 table of selection purchase and sale products	25
Table 3 File generation of purchase of products.	26
Table 4 file generation of sales process and products.	28
Table 5 To Show the update in stock of products.	30

1. INTRODUCTION:

The project for the coursework, named “WeCare products Wholesale System,” has been created to help WeCare, a local vendor focusing on the beauty and skincare products with efficient management of products inventory, customers sales and promotional campaigns effectively. As parts of its marketing strategy, WeCare offers a “Buy 3, Get 1 Free” deal on its products. This initiative seeks to simulate a basic wholesale management system that can manage sales, monitoring stock levels, adjusting prices and generating invoices for every transaction.

The goal is to design and develop a console-based application in python that imports product information from a well-structured text file, processes sales with flexible pricing, and automatically enforces promotional rules. The application then updates inventory based on sales and creates an invoice saved into a different file in same folder. Additionally, it provides the ability to restock items, enabling users to adjust both the quantity of products and their cost price and necessary.

Additionally, the system includes a feature for restocking products and modifying the products list, ensuring that the information remains up to date. The application determines the selling price by implementing a 200% markup based on the items cost price as indicated in the product file.

The primary aim is to develop a streamlined system that enables users to oversee inventory, handle sales, and produce essential invoices, while also offering the capability to restock and revise product information. The overall goal is to automate and enhance the processes of inventory management and sales transactions for WeCare.

1.1 Aims:

To develop a basic wholesale system using the python programming concepts. And to simulate product data handling from a structured text file.

1.2 Objectives:

- To create a .txt file that holds product details.
- To read data from the file using Python's file handling system.
- To keep the collected data in suitable data structures such as lists and dictionaries.
- To present the information in a clear table format through basic string formatting and print () function.
- To provide a simple text-based user interface featuring a main menu.

1.3 Technologies used:

- **Python:** Python runs seamlessly on Windows, macOS and Linux. In the WeCare product Wholesale system, python is used to arrange stock products, handle buying and selling operations, and generate invoices. The program is divided into 4 modules to make easy to use the code.

Python's basic features like loops, conditions, file handling, and functions are used to read product data from file, update stock, apply offers like "Buy 3 Get 1 Free," calculating selling prices, 200% markup, and save invoices. This makes the system efficient, user-friendly, and easy to use (geeksforgeeks, 2025).



Figure 1 python.

- **IDLE:**
It is a cross-platform works mostly the same on Windows, Unix, and macOS. IDLE offers an easy and beginner-friendly interface for writing, executing, and debugging Python programs. For this project, IDLE was used to develop and test the core Python logic of the WeCare Product Wholesale System, including file handling, calculations, and invoice generation. Its simplicity made it especially suitable for rapid testing and debugging in an academic setting (docs.python, 2025).



Figure 2. IDLE Python.

- **Draw.io:** For the drawing the flowchart of the WeCare products Wholesale system, I used to draw.io, a free and easy to use online drawing tool. It helped me visually plan the steps of the program such as reading product information, selling products, and providing the invoice. With draw.io, I was able to drag and drop shapes, connect them with arrows, and show the logic flow of my program in a clear manner. It helped me to better understand and explain how the system works (drawio, 2023).



Figure 3 draw.io

- **Microsoft Word:**

It is a popular word processing program and was created by the company. It offers powerful tools for formatting, editing, and structuring documents, making it ideal for preparing professional reports. In this project, Microsoft Word was used to the report of the Coursework such as heading, tables, image insertion, and referencing tools helped structure the report clearly and professionally (theknowledgeacademy, 2025).



Figure 4. Microsoft Word.

- **My Second Teacher (MST):**

My Second Teacher is an online education platform. For this project, MST was used to submit the Milestone report. The platform provides a structured interface where students can view submission deadlines, upload required documents.

2. Discuss and Analysis:

2.1 Algorithm:

Step 1: START

Step 2: Display store header details and current inventory.

Step 3: Ask the user whether they want to Sell Product, Buy Product, or Exit.

Step 4: If Sell Product, go to Step 5; if Buy Product, go to Step 12; else go to Step 19.

Step 5: Input customer name and phone number.

Step 6: Display current inventory details with selling price.

Step 7: Input product S.N and quantity to sell.

Step 8: Validate S.N and quantity, calculate free items using Buy 3 Get 1 Free offer.

Step 9: Add selected items to invoice list with total price.

Step 10: Ask if customer wants to continue buying, if yes go to Step 6.

Step 11: Generate and display sell invoice with timestamp, and save to invoice sell file.

Step 12: Ask vendor if they want to add new product or update existing.

Step 13: If adding new product, collect name, brand, quantity, price, and country.

Step 14: Assign new S.N and add the product to inventory.

Step 15: If updating existing product, input S.N and quantity to restock.

Step 16: Update product quantity and price in inventory.

Step 17: Save updated inventory to products.txt.

Step 18: Ask if vendor wants to restock more, if yes go to Step 12.

Step 19: Exit the program.

Step 20: END

2.2 Flowchart:

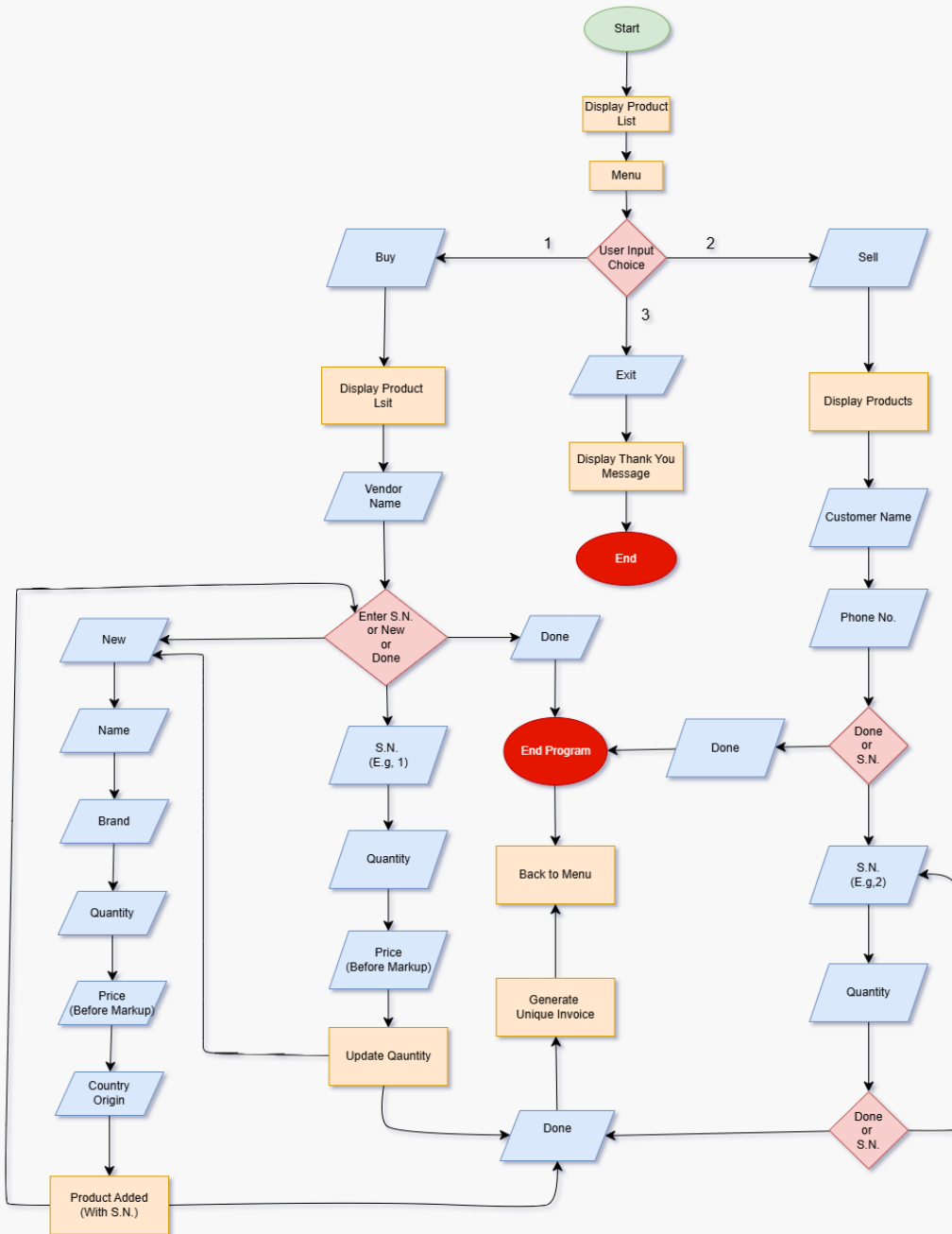


Figure 5. Flowchart diagram of the whole program.

2.3 Pseudocode:

#Read

```
BEGIN FUNCTION read_products
  TRY:
    OPEN "products.txt" to read .
    READ all line and store in list
    CLOSE file

  CATCH FileNotFoundError:
    DISPLAY "Error: 'products.txt'. not found"
    RETURN an empty list

  INITIALIZE products as empty list
  SET i = 1 // Skip the header row

  LOOP UNTIL i is less than length of lines:
    SET line = lines[i]

    INITIALIZE new_line as empty string
    FOR EACH character in line:
      IF character is not newline:
        APPEND character to new_line

    INITIALIZE parts as empty list
    INITIALIZE part as empty string

    FOR EACH character in new_line:
      IF character is comma:
        APPEND part to parts
        RESET part to empty string
      ELSE:
        APPEND character to part

    APPEND last part to parts

    IF length of parts is 6:
      CREATE product dictionary with keys:
        "sn", "name", "brand", "quantity", "price", "country"
      APPEND product to products.

    INCREMENT i by 1

  RETURN products
END FUNCTION
```

BEGIN FUNCTION display_products(products, show_markup=False)

DISPLAY separator line

DISPLAY table headers

DISPLAY separator line

INITIALIZE i = 0

LOOP UNTIL i is less than length of products:

SET p = products[i]

EXTRACT:

 name = p["name"]

 brand = p["brand"]

 qty = p["quantity"]

 sn = p["sn"]

 country = p["country"]

TRY:

IF show_markup is True:

CALCULATE price_val as p["price"] * 3

ELSE:

CALCULATE price_val as p["price"]

CATCH ValueError:

DISPLAY "Error reading product data. Skipping item."

INCREMENT i

CONTINUE LOOP

FORMAT:

 price = "NRP " + price_val

 // Apply padding for table alignment

WHILE length of name is less than 18:

APPEND space to name

WHILE length of brand is less than 18:

APPEND space to brand

WHILE length of qty is less than 5:

APPEND space to qty

WHILE length of price is less than 8:

APPEND space to price

WHILE length of country is less than 12:

APPEND space to country

IF length of sn is1:
 APPEND space to sn

DISPLAY the formatted row of product data

INCREMENT i

DISPLAY separator line
END FUNCTION

#Write

BEGIN FUNCTION write_products(products)

TRY:

OPEN "products.txt" in file.

WRITE header "SN,Name,Brand,Quantity,Price,Country" to file

FOR EACH product p in products:

FORMAT line as:

 sn, name, brand, quantity, price, country separated by commas

ADD newline character to the end of the line

WRITE the line to file

CATCH Exception as e:

DISPLAY "Error while writing file",e

END FUNCTION

BEGIN FUNCTION save_invoice(invoice_text, filename)

 """"

 Saves the invoice text to a .txt file.

 """"

TRY:

OPEN file in write mode.

WRITE invoice_text to the file

CATCH Exception as e:

DISPLAY "Error saving invoice:", e

END FUNCTION

#Operation**BEGIN FUNCTION** buy_product**CALL** read.read_products() and store in products**IF** products is empty:**RETURN****CALL** read.display_products(products)**PROMPT** "Enter vendor name" and store in vendor_name**INITIALIZE:**

selected = []

total = 0

successful_transaction = False

LOOP UNTIL user enters "done":**PROMPT** "Enter S.N of product to restock (or 'new' to add new product, or 'done' to finish)" and store in sn**IF** sn IS "done":**BREAK LOOP****ELSE IF** sn IS "new":**PROMPT** "Enter product name" and store in name**PROMPT** "Enter brand" and store in brand**PROMPT** "Enter quantity to add" and store in quantity**PROMPT** "Enter cost price" and store in price**PROMPT** "Enter country of origin" and store in country**GENERATE** new s.n as (length of products + 1)**CREATE** new_product dictionary with details**ADD** new_product to products**ADD** [name, brand, quantity, price, total price] to selected**ADD** total price to total**DISPLAY** "New product added"**SET** successful_transaction = True**ELSE:****SET** found = False**FOR EACH** product in products:


```
    IF product s.n matches sn:
        PROMPT "Enter quantity to add" and store in qty
        PROMPT "Enter updated cost price" and store in new_price

        UPDATE product quantity and price
        CALCULATE amount = qty * new_price

        ADD [product details, amount] to selected
        ADD amount to total
        DISPLAY "Product restocked with new quantity"
        SET successful_transaction = True
        SET found = True
        BREAK LOOP

    IF found IS False:
        DISPLAY "S.N not found"

IF successful_transaction is True:
    CALL write.write_products(products)

    GENERATE current timestamp
    GENERATE random voucher number
    CREATE invoice file name with timestamp and voucher number
    FORMAT invoice details

    FOR EACH item IN selected:
        FORMAT item details INTO invoice

    APPEND total amount TO invoice
    DISPLAY invoice details
    SAVE invoice file USING write.save_invoice()
    DISPLAY voucher number

END FUNCTION

BEGIN FUNCTION sell_product
    CALL read.read_products() and store in products

    IF products IS EMPTY:
        RETURN

    CALL read.display_products(products, show_markup=True)

    LOOP UNTIL valid customer name is entered:
        PROMPT "Enter customer name" and store in customer_name
        IF customer_name is not alphabetic:
```

```
    DISPLAY "Invalid customer name"  
ELSE:  
    BREAK LOOP
```

```
LOOP UNTIL valid phone number is entered:  
    PROMPT "Enter customer phone number" and store in customer_phone  
    IF customer_phone is not 10 digits:  
        DISPLAY "Invalid phone number"  
    ELSE:  
        BREAK LOOP
```

```
INITIALIZE:  
    selected = []  
    total = 0  
    successful_transaction = False  
    item_count = {} // For tracking how many are sold
```

```
LOOP UNTIL user enters "done":  
    PROMPT "Enter S.N of product to sell (or 'done') and store in sn
```

```
    IF sn IS "done":  
        BREAK LOOP
```

```
    SET found = False
```

```
    FOR EACH product in products:  
        IF product s.n matches sn:  
            PROMPT "Enter quantity to sell" and store in qty  
            CHECK product quantity in stock
```

```
        IF sn not in item_count:  
            SET item_count[sn] = 0
```

```
        ADD qty TO item_count[sn]
```

```
    CALCULATE:  
        free = item_count[sn] // 3  
        total_items = qty + free
```

```
    IF total_items > available stock:  
        DISPLAY "Not enough stock"  
        BREAK LOOP
```

```
    UPDATE product quantity in stock  
    CALCULATE total amount
```

```
    ADD product details to selected
    ADD amount to total
    SET successful_transaction = True
    SET found = True
    BREAK LOOP
```

```
IF found is False:
    DISPLAY "S.N not found"
```

```
IF successful_transaction is True:
    CALL write.write_products(products)
    DISPLAY "Transaction Completed Successfully!"
```

```
GENERATE current timestamp
GENERATE random voucher number
CREATE invoice file name with timestamp and voucher number
FORMAT invoice details
```

```
FOR EACH item in selected:
    FORMAT item details into invoice
```

```
APPEND total amount to invoice
DISPLAY invoice details
SAVE invoice file using write.save_invoice()
DISPLAY voucher number
```

```
END FUNCTION
```

Main.py

```
BEGIN MAIN_PROGRAM
```

```
OUTPUT "WELCOME TO WECARE WHOLESALE SYSTEM"
OUTPUT "-----"
OUTPUT "Address: Bagbazar, Kathmandu | Ph. 9821000760"
OUTPUT "-----"
```

```
LOOP INDEFINITELY
```

```
    TRY
```

```
        CALL read.read_products() and store in products
        CALL read.display_products(products, show_markup=True)
```

```
        OUTPUT "Menu:"
        OUTPUT "1. Buy Product (Restock)"
        OUTPUT "2. Sell Product"
        OUTPUT "3. Exit"
```

```
PROMPT "Enter choice (1/2/3):  
  
IF choice == "1" then  
    CALL operation.buy_product()  
ELSE IF choice == "2" then  
    CALL operation.sell_product()  
ELSE IF choice == "3" then  
    OUTPUT "Thank you for using WeCare Wholesale System"  
    BREAK LOOP  
ELSE  
    OUTPUT "Invalid choice."  
END IF  
  
EXCEPT Exception as err  
    OUTPUT "invalid choice: ", err  
END TRY  
END LOOP  
  
END MAIN_PROGRAM
```

2.4 Data Structure:

List: List in Python is a collection of items stored in a single variable. Lists in python are indexed starting from 0, meaning each element can be accessed using its position in the list. List don't use keys, instead they use indices to access the elements. List can store any type of data like numbers, strings, or even other lists.

Example: ["facewash", "Shampoo", "Sunscreen"]

```
try:
    with open("products.txt", "w") as file:
        file.write("SN,Name,Brand,Quantity,Price,Country\n")
        for p in products:
            line = (
                p['sn'] + "," +
                p['name'] + "," +
                p['brand'] + "," +
                str(p['quantity']) + "," +
                str(p['price']) + "," +
                p['country'] + "\n"
            )
            file.write(line)
except Exception as e:
    print("Error writing to file: " + str(e))
```

Figure 6 screen shot of the list example.

Dictionaries: Dictionary is a combination of key-values pairs, where every key is unique and each key maps to a value. Unlike lists, dictionaries use keys instead of indices to store and retrieve data.

Example: {"name": "Facewash", "Price": 100, "quantity":50}

```

new_product = {
    "sn": new_sn,
    "name": name,
    "brand": brand,
    "quantity": quantity,
    "price": price,
    "country": country
}
products.append(new_product)
selected.append([name, brand, quantity, price, quantity * price])
total += quantity * price
print("New product '" + name + "' added with S.N " + new_sn + ".")
successful_transaction = True

```

Figure 7 screen shot of the dictionary eg.

String: A string is a group of characters used to hold text in computer programming. It can hold letters, numbers, symbols, and even spaces. String in python are used for the textual data. We use them for representing names, countries, invoices, and other textual information in my code.

Example: Product name = "Vitamin C Serum"

```

if successful_transaction:
    now = datetime.datetime.now()
    voucher = str(random.randint(1000, 9999))
    fname = "invoice_buy_" + voucher + "_" + now.strftime('%Y%m%d%H%M') + ".txt"
    invoice = "Voucher No: " + voucher + "\nVendor Name: " + vendor_name + "\nDate: "
    invoice += "Product      Brand      Qty      Price      Amount\n"

```

Figure 8 screen shot for the string eg.

Tuple: Tuple is a data structure similar to a list, but unlike lists, tuples are immutable.

Once a tuple is created, you cannot modify its contents.

Example: Selected. append([name, brand, quantity, price, quantity*price])

```

}
products.append(new_product)
selected.append([name, brand, quantity, price, quantity * price])
total += quantity * price
print("New product '" + name + "' added with S.N " + new_sn + ".")
successful_transaction = True

```

Figure 9 screen shot for the tuple eg..

Integer: Integers represent whole numbers and are used in my code to represent quantities, prices, and serial numbers.

Example: S.N = 1, Qty = 100 etc.

```
try:
    available = int(p["quantity"])
    if sn not in item_count:
        item_count[sn] = 0
    item_count[sn] += qty
```

Figure 10 screen shot for the integer eg.

3. Program:

The program is a Python-based wholesale management system for WeCare. It's divided into 4 modules:

- Main.py: Handles the menu and program loop.
- Operation.py: It contains functions for buying and selling products.
- Read.py: It handles the file reading.
- Write.py: It handles the file writing.

And a products.txt file which store products data persistently.

The system performs three main tasks:

1. Buy Product (Restock)
2. Sell product (with "Buy 3 Get 1 Free")
3. Exit

The buy and sell actions update the product inventory and generate a voucher/invoices as a .txt file.

Showing the buy and sale of the product (Complete process):**1. Buy process:**

- User chooses Buy product.
- Current products are displayed from the product.txt file.
- User can restock an existing product by entering S.N Or add a new product with its details.
- The selected products are stored with full details.
- Increase the inventory based on the item added.
- Once done, a buy invoice is generated and printed like this:

```
Voucher No: 4301
Vendor Name: wasi
Date: 2025-05-13 02:19

Product      Brand      Qty  Price  Amount
Vitamin C SerumGarnier    5    1000   5000

Total Amount: 5000
Voucher Number (Buy): 4301
```

Figure 11 voucher for buy.

This invoice is saved in a file like: invoice_buy_4301_202505130219.txt

```

members_dæ invoice_sell_ java code dæ milestæ • Simple.txt
File Edit View

Voucher No: 4301
Vendor Name: wasi
Date: 2025-05-13 02:19

Product      Brand      Qty  Price  Amount
Vitamin C SerumGarnier  5    1000   5000

Total Amount: 5000

```

Figure 12 screen shot of buy invoice txt file.

```

Menu:
1. Buy Product (Restock)
2. Sell Product
3. Exit
Enter Your choice (1/2/3): 1

| S.N | Product Name      | Brand Name      | Qty | Price | Country |
|-----|-----|-----|-----|-----|-----|
| 1 | Vitamin C Serum | Garnier         | 200 | NRP 1000 | France |
| 2 | Skin Cleanser   | Cetaphil        | 90  | NRP 280  | Switzerland |
| 3 | Sunscreen       | Aqualogica      | 187 | NRP 700  | India   |
| 4 | Facewash        | Dr Rashel       | 282 | NRP 500  | UAE     |
| 5 | Shampoo         | Mamaearth       | 243 | NRP 550  | India   |
| 6 | Moisturizer     | Joy             | 284 | NRP 250  | India   |
| 7 | Suncream        | Joy             | 150 | NRP 200  | India   |
|-----|-----|-----|-----|-----|-----|

Enter vendor name: wasi
Enter S.N of product to restock (or 'new' to add new product, or 'done' to finish): 1
Enter quantity to add: 5
Enter updated cost price (before markup): 1000
Product 'Vitamin C Serum' restocked. New quantity: 205
Enter S.N of product to restock (or 'new' to add new product, or 'done' to finish): done
Voucher No: 4301
Vendor Name: wasi
Date: 2025-05-13 02:19

Product      Brand      Qty  Price  Amount
Vitamin C SerumGarnier  5    1000   5000

Total Amount: 5000
Voucher Number (Buy): 4301

```

Figure 13 complete output for the buy.

2. Sale process:

- User selects sell products.
- Product list with markup price with 200% is shown.
- User enters customer name and phone number.
- For each products, enter S.N and quantity.
- Automatically applies “Buy 3 Get 1 Free” by calculating free items.
- Calculate the markup price .
- Reduces the inventory based on the item sold + free items.
- Generates the invoice like this:

```
Transaction Completed Successfully!

Voucher No: 7721
Customer Name: wasi
Phone Number: 9800000000
Date: 2025-05-13 02:32:07

Product      Brand      Qty  Free  Price  Amount
Vitamin C    SerumGarnier    3    1    3000   9000

Total Amount: 9000
Voucher Number (Sell): 7721
```

Figure 14 screen shot of invoice for sell.

This is saved in a file like: invoice_sell_7721_202505130232.txt

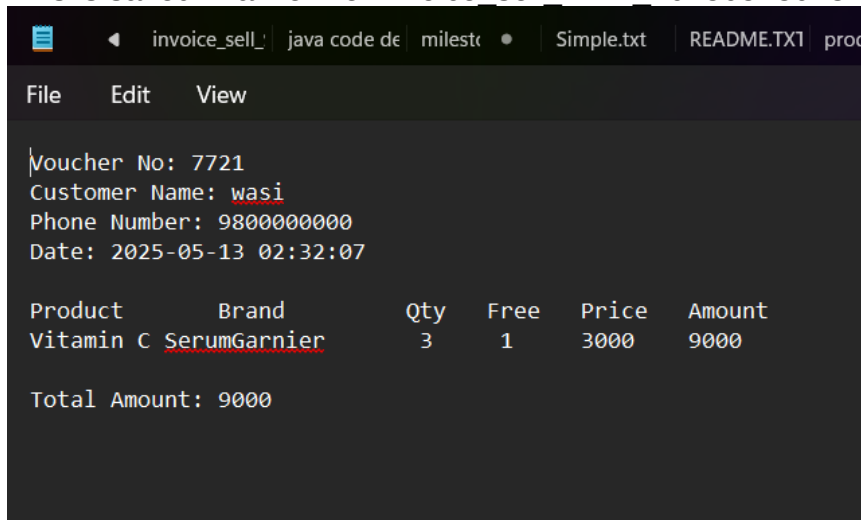


Figure 15 screen shot for the sell txt.

```

Menu:
1. Buy Product (Restock)
2. Sell Product
3. Exit
Enter Your choice (1/2/3): 2

```

S.N	Product Name	Brand Name	Qty	Price	Country
1	Vitamin C Serum	Garnier	205	NRP 3000	France
2	Skin Cleanser	Cetaphil	90	NRP 840	Switzerland
3	Sunscreen	Aqualogica	187	NRP 2100	India
4	Facewash	Dr Rashel	282	NRP 1500	UAE
5	Shampoo	Mamaearth	243	NRP 1650	India
6	Moisturizer	Joy	284	NRP 750	India
7	Suncream	Joy	150	NRP 600	India

```

Enter customer name: wasi
Enter customer phone number (10 digits): 9800000000
Enter S.N of product to sell (or 'done'): 1
Enter quantity to sell: 3
Product 'Vitamin C Serum' sold successfully with 1 free items.
Enter S.N of product to sell (or 'done'): done

Transaction Completed Successfully!

Voucher No: 7721
Customer Name: wasi
Phone Number: 9800000000
Date: 2025-05-13 02:32:07

Product      Brand      Qty  Free  Price  Amount
Vitamin C SerumGarnier    3    1    3000    9000

Total Amount: 9000
Voucher Number (Sell): 7721

```

Figure 16 complete output for sell.

3. Creation and Opening of txt file and showing bill:

- Invoice filenames are created using:

```
fname = "invoice_sell_" + voucher + "_" + now.strftime('%Y%m%d%H%M') + ".txt"
```
- They are saved using `write.save_invoice(invoice, fname)`.
- The bill is printed to the screen and saved as a .txt file for future records.

4.Termination of the program:

- When user selects option 3 (Exit) from the menu:

```

WELCOME TO WECARE WHOLESALE SYSTEM
-----

Address: Bagbazar, Kathmandu | Ph. 9821000760
-----

| S.N | Product Name | Brand Name | Qty | Price | Country |
-----|-----|-----|-----|-----|-----|
| 1 | Vitamin C Serum | Garnier | 201 | NRP 3000 | France |
| 2 | Skin Cleanser | Cetaphil | 90 | NRP 840 | Switzerland |
| 3 | Sunscreen | Aqualogica | 187 | NRP 2100 | India |
| 4 | Facewash | Dr Rashel | 282 | NRP 1500 | UAE |
| 5 | Shampoo | Mamaearth | 243 | NRP 1650 | India |
| 6 | Moisturizer | Joy | 284 | NRP 750 | India |
| 7 | Suncream | Joy | 150 | NRP 600 | India |
-----

Menu:
1. Buy Product (Restock)
2. Sell Product
3. Exit
Enter Your choice (1/2/3): 3
Thank you for using WeCare Wholesale System
>>

```

Figure 17 exit program.

- This cleanly end the loop and exits the program.

4. Testing:

4.1 Test 1: Showing implementation of try and except:

Objective	To show the implement of try, except
Action	Provide a non-numeric input such as “aaa” for options.
Expected Result	Program should catch the Value Error and display a friendly error message.
Actual Result	Displayed invalid choice
Conclusion	The test is successful.

Table 1 Test to show the implementation of try except

```
Menu:
1. Buy Product (Restock)
2. Sell Product
3. Exit
Enter Your choice (1/2/3): aaa
Invalid choice.
```

Figure 18 screen shot of the output after entering non numeric value.

4.2 Test 2: Selection purchase and sale products

Objective	To provide a negative value as input
Action	- Enter -5 as quantity during buying/selling. - Enter 99 as input (as non-existed value)
Expected Result	- Program should reject the input or result in logical failure. - Display message "invalid choice."
Actual Result	- After providing a negative value as input, program reject the input and shows this message "Quantity must be a positive integer. Please try again" is displayed. - After providing a non-existed value the message "invalid choice" is displayed.
Conclusion	The test is successful.

Table 2 table of selection purchase and sale products

```

Enter vendor name: wasi
Enter S.N of product to restock (or 'new' to add new product, or 'done' to finish): 1
Enter quantity to add: -5
Quantity must be a positive integer. Please try again
Enter S.N of product to restock (or 'new' to add new product, or 'done' to finish): done

```

Figure 19 Screenshot of providing negative input.

```

Menu:
1. Buy Product (Restock)
2. Sell Product
3. Exit
Enter Your choice (1/2/3): 99
Invalid choice.

```

Figure 20 screen shot of providing non-existed value.

4.3 Test 3: File generation of purchase of product(s)

Objective	To show file generation of purchase of products.
Action	Purchase multiple products using valid S.Ns.
Expected Result	Purchase invoice generated with product details in shell and saved as invoice_buy_voucher_number_timestamp.txt
Actual Result	Successfully displayed the invoice and saved file with correct format and timestamp.
Conclusion	The test is successful.

Table 3 File generation of purchase of products.

```

WELCOME TO WECARE WHOLESALE SYSTEM
-----
Address: Bagbazar, Kathmandu | Ph. 9821000760
-----

| S.N | Product Name | Brand Name | Qty | Price | Country |
|-----|-----|-----|-----|-----|-----|
| 1 | Vitamin C Serum | Garnier | 208 | NRP 3000 | France |
| 2 | Skin Cleanser | Cetaphil | 90 | NRP 840 | Switzerland |
| 3 | Sunscreen | Aqualogica | 187 | NRP 2100 | India |
| 4 | Facewash | Dr Rashel | 282 | NRP 1500 | UAE |
| 5 | Shampoo | Mamaearth | 243 | NRP 1650 | India |
|-----|-----|-----|-----|-----|

Menu:
1. Buy Product (Restock)
2. Sell Product
3. Exit
Enter Your choice (1/2/3): 1

| S.N | Product Name | Brand Name | Qty | Price | Country |
|-----|-----|-----|-----|-----|-----|
| 1 | Vitamin C Serum | Garnier | 208 | NRP 1000 | France |
| 2 | Skin Cleanser | Cetaphil | 90 | NRP 280 | Switzerland |
| 3 | Sunscreen | Aqualogica | 187 | NRP 700 | India |
| 4 | Facewash | Dr Rashel | 282 | NRP 500 | UAE |
| 5 | Shampoo | Mamaearth | 243 | NRP 550 | India |
|-----|-----|-----|-----|-----|

Enter vendor name: wasiur
Enter S.N of product to restock (or 'new' to add new product, or 'done' to finish): 1
Enter quantity to add: 12
Enter updated cost price(before markup)1000
Product 'Vitamin C Serum' restocked. New quantity: 220
Enter S.N of product to restock (or 'new' to add new product, or 'done' to finish): 5
Enter quantity to add: 7
Enter updated cost price(before markup)550
Product 'Shampoo' restocked. New quantity: 250
Enter S.N of product to restock (or 'new' to add new product, or 'done' to finish): done

```

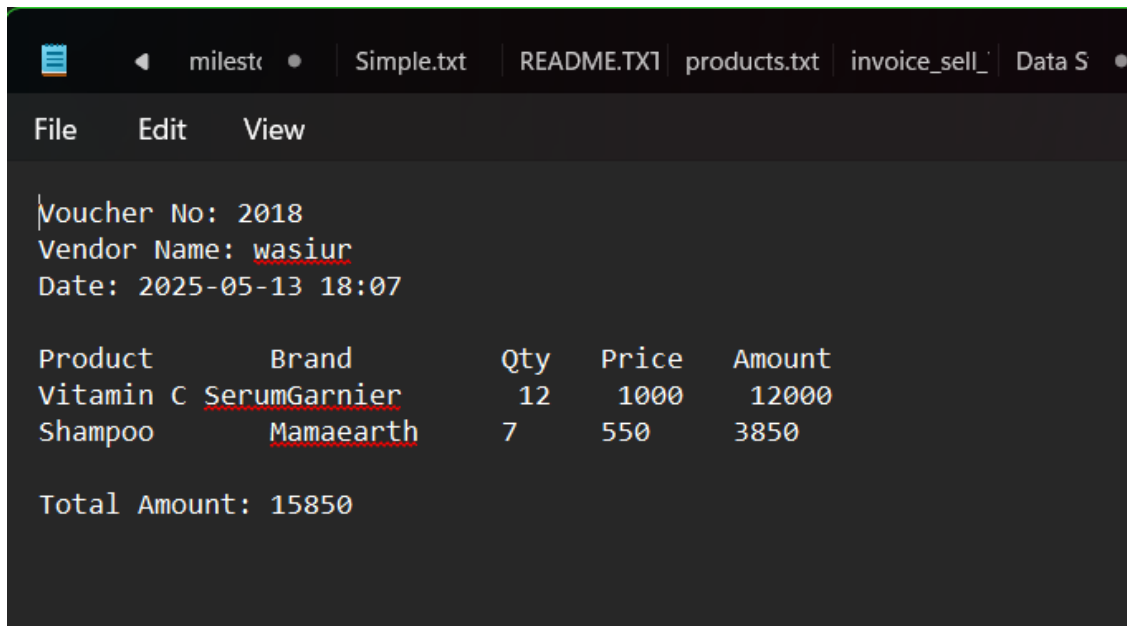
Figure 21 complete purchase process.

Voucher No: 2018
Vendor Name: wasiur
Date: 2025-05-13 18:07

Product	Brand	Qty	Price	Amount
Vitamin C Serum	Garnier	12	1000	12000
Shampoo	Mamaearth	7	550	3850

Total Amount: 15850
Voucher Number (Buy): 2018

Figure 22 output in the sell .



```
Voucher No: 2018
Vendor Name: wasiur
Date: 2025-05-13 18:07

Product      Brand      Qty  Price  Amount
Vitamin C SerumGarnier    12   1000   12000
Shampoo      Mamaearth  7     550   3850

Total Amount: 15850
```

Figure 23 purchased products details in a text file.

4.4 Test 4: File generation of sales process of product(s).

Objective	To show file generation of sales process of products.
Action	Sell multiple products with S.N and quantity.
Expected Result	Generates and display sales invoice in the shell and save as invoice_sell_voucher_number_timestamp.txt
Actual Result	Invoice saved and contained fields, product, quantity, free, price, amount.
Conclusion	The test is successful.

Table 4 file generation of sales process and products.

```

WELCOME TO WECARE WHOLESALE SYSTEM
-----
Address: Bagbazar, Kathmandu | Ph. 9821000760
-----

| S.N | Product Name      | Brand Name      | Qty | Price      | Country      |
|-----|-----|-----|-----|-----|-----|
| 1 | Vitamin C Serum   | Garnier         | 220 | NRP 3000   | France       |
| 2 | Skin Cleanser     | Cetaphil        | 90  | NRP 840    | Switzerland  |
| 3 | Sunscreen         | Aqualogica      | 187 | NRP 2100   | India        |
| 4 | Facewash          | Dr Rashel       | 282 | NRP 1500   | UAE          |
| 5 | Shampoo           | Mamaearth       | 250 | NRP 1650   | India        |
|-----|-----|-----|-----|-----|-----|

Menu:
1. Buy Product (Restock)
2. Sell Product
3. Exit
Enter Your choice (1/2/3): 2

| S.N | Product Name      | Brand Name      | Qty | Price      | Country      |
|-----|-----|-----|-----|-----|-----|
| 1 | Vitamin C Serum   | Garnier         | 220 | NRP 3000   | France       |
| 2 | Skin Cleanser     | Cetaphil        | 90  | NRP 840    | Switzerland  |
| 3 | Sunscreen         | Aqualogica      | 187 | NRP 2100   | India        |
| 4 | Facewash          | Dr Rashel       | 282 | NRP 1500   | UAE          |
| 5 | Shampoo           | Mamaearth       | 250 | NRP 1650   | India        |
|-----|-----|-----|-----|-----|-----|

Enter customer name: wasiurrahman
Enter customer phone number (10 digits): 9898009898
Enter S.N of product to sell (or 'done'): 3
Enter quantity to sell: 7
Product 'Sunscreen' sold successfully with 2 free items.
Enter S.N of product to sell (or 'done'): 4
Enter quantity to sell: 2
Product 'Facewash' sold successfully with 0 free items.
Enter S.N of product to sell (or 'done'): done

```

Figure 24 output of entire process during sell.

```

Transaction Completed Successfully!

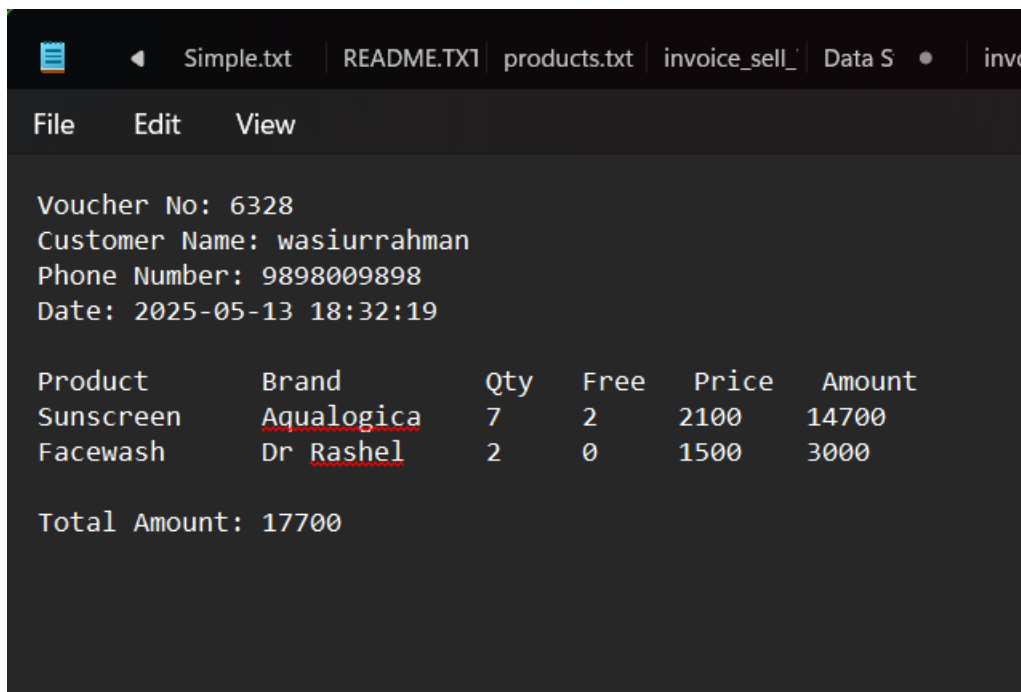
Voucher No: 6328
Customer Name: wasiurrahman
Phone Number: 9898009898
Date: 2025-05-13 18:32:19

Product      Brand      Qty  Free  Price  Amount
Sunscreen    Aqualogica  7    2    2100   14700
Facewash     Dr Rashel   2    0    1500   3000

Total Amount: 17700
Voucher Number (Sell): 6328

```

Figure 25 output of invoice generated in shell during sell.



The screenshot shows a text editor window with several tabs: 'Simple.txt', 'README.TXT', 'products.txt', 'invoice_sell_', 'Data S', and 'inv...'. The 'invoice_sell_' tab is active, displaying the following text:

```

Voucher No: 6328
Customer Name: wasiurrahman
Phone Number: 9898009898
Date: 2025-05-13 18:32:19

Product      Brand      Qty  Free  Price  Amount
Sunscreen    Aqualogica  7    2    2100   14700
Facewash     Dr Rashel   2    0    1500   3000

Total Amount: 17700

```

Figure 26 output of invoice generated in txt.

4.5 Test 5: Showing the update in stock of product(s)

Objective	- To show the updates in stock during purchase - To show the updates in stock during sell
Action	- Restock product with S.N 1 by quantity 10. - Sale product with S.N 2 with quantity 6.
Expected Result	- Product quantity should increase by 10 in the stock file. - Product quantity should reduce by 6 in stock file
Actual Result	- Confirmed update in products.txt file. - Verified correct deduction in stock.
Conclusion	The test is Successful.

Table 5 To Show the update in stock of products.

For Buy Products:

```

Menu:
1. Buy Product (Restock)
2. Sell Product
3. Exit
Enter Your choice (1/2/3): 1

```

S.N	Product Name	Brand Name	Qty	Price	Country
1	Vitamin C Serum	Garnier	220	NRP 1000	France
2	Skin Cleanser	Cetaphil	90	NRP 280	Switzerland
3	Sunscreen	Aqualogica	178	NRP 700	India
4	Facewash	Dr Rashel	280	NRP 500	UAE
5	Shampoo	Mamaearth	250	NRP 550	India

```

Enter vendor name: wasi
Enter S.N of product to restock (or 'new' to add new product, or 'done' to finish): 1
Enter quantity to add: 10
Enter updated cost price(before markup)1000
Product 'Vitamin C Serum' restocked. New quantity: 230
Enter S.N of product to restock (or 'new' to add new product, or 'done' to finish): done
Voucher No: 5793
Vendor Name: wasi
Date: 2025-05-13 21:38

Product      Brand      Qty  Price  Amount
Vitamin C SerumGarnier    10   1000   10000

Total Amount: 10000
Voucher Number (Buy): 5793

WELCOME TO WECARE WHOLESALE SYSTEM
-----
Address: Bagbazar, Kathmandu | Ph. 9821000760
-----

```

S.N	Product Name	Brand Name	Qty	Price	Country
1	Vitamin C Serum	Garnier	230	NRP 3000	France
2	Skin Cleanser	Cetaphil	90	NRP 840	Switzerland
3	Sunscreen	Aqualogica	178	NRP 2100	India
4	Facewash	Dr Rashel	280	NRP 1500	UAE
5	Shampoo	Mamaearth	250	NRP 1650	India

Figure 27 stock update after purchasing.

```

Voucher No: 5793
Vendor Name: wasi
Date: 2025-05-13 21:38

Product      Brand      Qty  Price  Amount
Vitamin C SerumGarnier  10   1000   10000

Total Amount: 10000

```

Figure 28 screen shot of update stock in txt file of buy product.

For sell products:

```

2. Sell Product
3. Exit
Enter Your choice (1/2/3): 2

| S.N | Product Name | Brand Name | Qty | Price | Country |
|-----|-----|-----|-----|-----|-----|
| 1 | Vitamin C Serum | Garnier | 230 | NRP 3000 | France |
| 2 | Skin Cleanser | Cetaphil | 90 | NRP 840 | Switzerland |
| 3 | Sunscreen | Aqualogica | 178 | NRP 2100 | India |
| 4 | Facewash | Dr Rashel | 280 | NRP 1500 | UAE |
| 5 | Shampoo | Mamaearth | 250 | NRP 1650 | India |

Enter customer name: wasi
Enter customer phone number (10 digits): 9898009800
Enter S.N of product to sell (or 'done'): 1
Enter quantity to sell: 6
Product 'Vitamin C Serum' sold successfully with 2 free items.
Enter S.N of product to sell (or 'done'): done

Transaction Completed Successfully!

Voucher No: 3987
Customer Name: wasi
Phone Number: 9898009800
Date: 2025-05-13 22:48:40

Product      Brand      Qty  Free  Price  Amount
Vitamin C SerumGarnier  6     2    3000   18000

Total Amount: 18000
Voucher Number (Sell): 3987

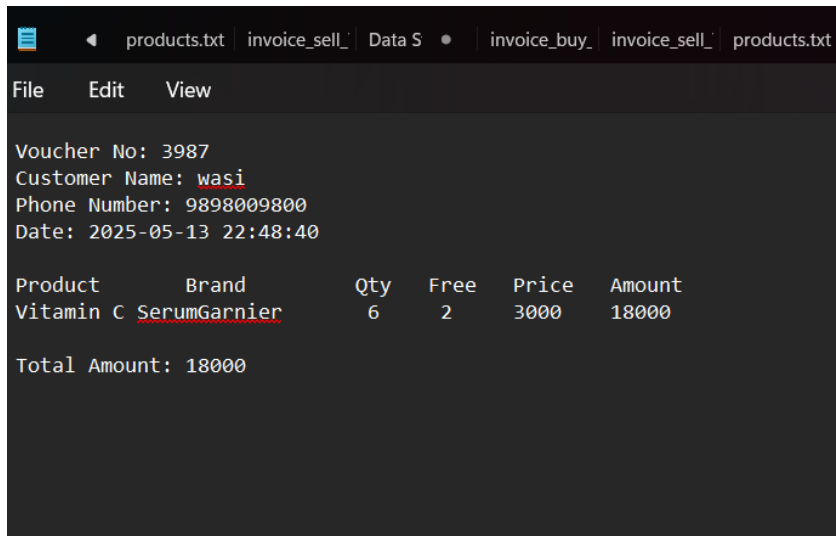
WELCOME TO WECARE WHOLESALE SYSTEM
-----

Address: Bagbazar, Kathmandu | Ph. 9821000760
-----

| S.N | Product Name | Brand Name | Qty | Price | Country |
|-----|-----|-----|-----|-----|-----|
| 1 | Vitamin C Serum | Garnier | 222 | NRP 3000 | France |
| 2 | Skin Cleanser | Cetaphil | 90 | NRP 840 | Switzerland |
| 3 | Sunscreen | Aqualogica | 178 | NRP 2100 | India |
| 4 | Facewash | Dr Rashel | 280 | NRP 1500 | UAE |
| 5 | Shampoo | Mamaearth | 250 | NRP 1650 | India |

```

Figure 29 Stock updated after sell.



The screenshot shows a text editor window with a dark theme. The title bar at the top displays several open files: products.txt, invoice_sell_, Data S, invoice_buy_, invoice_sell_, and products.txt. The menu bar includes File, Edit, and View. The main text area contains the following information:

```
Voucher No: 3987
Customer Name: wasi
Phone Number: 9898009800
Date: 2025-05-13 22:48:40
```

Product	Brand	Qty	Free	Price	Amount
Vitamin C	SerumGarnier	6	2	3000	18000

Total Amount: 18000

Figure 30 Screenshot stock updated in txt file after sell.

5. Conclusion:

The WeCare Wholesale system is a simple but efficient python program that is designed to manage buying and selling of skin care items for a local vendor. An administrator using this system can manage product stock effectively, apply promotion discounts such as “Buy 3, Get 1 Free” and generate customer invoices with accurate pricing and quantity control. The system reads product data from a file, keeps inventory after each transaction, and enforces data correctness using user input validation and exception handling.

Throughout the project, we used a beginner-level approach by only using essential python constructor such as loops, conditions, functions, and file handling. The code was organized into four independent modules like read.py, write.py, operation.py, and main.py which is modular programming and separation of concerns. Each module had a specific role, such as reading product data, updating files, executing business logic, and managing the user interface. Exception handling was incorporated to prevent the program

from crashing despite improper input, particularly for data types and stock availability of products.

Some of the take home results were about how inventory system operate in retailing how they determine profit margins by 200% markup and how they affects stock and billing on promotions like “Buy 3 Get 1 Free”. We also learned about the necessity of producing invoices with unique filenames, by timestamp, to track transactions properly. Another wonderful insight was about how input validation can make such systems much more reliable and amenable to using.

The project not only helped reinforce programming skills but also provided practical exposure to problem solving, user input handling, and real life system design. It concluded with a fully functional, text based wholesale system capable of managing products, applying promotions, and generating accurate invoices. Overall, the system is reliable, easy to use, and can serve as a strong foundation for more advanced systems with graphical interfaces or database integration in the future.

Bibliography

1. docs.python, 2025. *docs.python*. [Online]
Available at: <https://docs.python.org/3/library/idle.html>
[Accessed 13 may 2025].
2. drawio, 2023. *draw.io*. [Online]
Available at: <https://www.drawio.com/about>
[Accessed 13 may 2025].
3. geeksforgeeks, 2025. *geeksforgeeks*. [Online]
Available at: <https://www.geeksforgeeks.org/introduction-to-python/>
[Accessed 13 may 2025].
4. theknowledgeacademy, 2025. *theknowledgeacademy*. [Online]
Available at: <https://www.theknowledgeacademy.com/blog/what-is-microsoft-word/>
[Accessed 13 may 2025].

6. Appendix:

Write.py

----- Write Functions -----

```
def write_products(products):
```

```
    """
```

Writes the updated list of products to the file 'products.txt'.

This function overwrites the existing content in 'products.txt' with the new list of products, formatted in CSV style. It includes a header row followed by the product details.

Parameters:

products : list

A list of dictionaries, each representing a product.

Required keys: 'sn', 'name', 'brand', 'quantity', 'price', 'country'.

Returns:

None

Raises:

Exception:

If an error occurs while writing to 'products.txt', it is caught and printed.

```
    """
```

```
    try:
```

```
        # Open the file in write mode
```

```
        with open("products.txt", "w") as file:
```

```
            # Write the header
```

```
            file.write("SN,Name,Brand,Quantity,Price,Country\n")
```

```
        # Write each product's data
```

```
        for p in products:
```

```
            # Manually concatenate the line
```

```
            line = (
```

```
                p['sn'] + "," +
```

```
                p['name'] + "," +
```

```
                p['brand'] + "," +
```

```
                str(p['quantity']) + "," +
```

```
                str(p['price']) + "," +
```

```
                p['country'] + "\n"
```



```

    )
    # Write to the file
    file.write(line)
except Exception as e:
    # Print the error if something goes wrong
    print("Error writing to file: " + str(e))

```

```
def save_invoice(invoice_text, filename):
```

```
    """
```

Saves the generated invoice to a text file with the specified filename.

This function creates or overwrites a text file with the provided invoice content.

Parameters:

```
-----
```

invoice_text : str

The complete invoice text to be written to the file.

filename : str

The name of the file where the invoice will be saved.

Returns:

```
-----
```

None

Raises:

```
-----
```

Exception:

If an error occurs while saving the invoice, it is caught and printed.

```
    """
```

```
try:
```

```
    # Open the file in write mode
```

```
    with open(filename, "w") as file:
```

```
        # Write the invoice text to the file
```

```
        file.write(invoice_text)
```

```
except Exception as e:
```

```
    # Print the error if something goes wrong
```

```
    print("Error saving invoice: " + str(e))
```

Read.py

```
# ----- Read Functions -----
```

```
def read_products():
    """
```

Reads the products from the 'products.txt' file and returns them as a list of dictionaries.

Each line in the file is expected to contain the following comma-separated fields:
S.N, Product Name, Brand Name, Quantity, Price, Country of Origin

The function skips the header and converts each line into a dictionary, which is then added to a list.

Parameters:

None

Returns:

list:

A list of dictionaries representing the products.

Each dictionary contains the keys: 'sn', 'name', 'brand', 'quantity', 'price', 'country'.

Raises:

FileNotFoundError:

If 'products.txt' is not found, it prints an error and returns an empty list.

"""

try:

file = open("products.txt", "r")

lines = file.readlines()

file.close()

except FileNotFoundError:

print("Error: 'products.txt' file not found.")

return []

products = []

i = 1

while i < len(lines): # skip header line

line = lines[i]

new_line = ""

j = 0

Manually strip the newline characters

while j < len(line):

if line[j] != "\n":

new_line = new_line + line[j]

j = j + 1

```

# Split the line by commas manually
parts = []
part = ""
k = 0
while k < len(new_line):
    if new_line[k] == ",":
        parts.append(part)
        part = ""
    else:
        part = part + new_line[k]
    k = k + 1
parts.append(part)

# Only append if all required fields are available
if len(parts) == 6:
    product = {
        "sn": parts[0],
        "name": parts[1],
        "brand": parts[2],
        "quantity": parts[3],
        "price": parts[4],
        "country": parts[5]
    }
    products.append(product)
    i = i + 1

return products

```

----- Display Function -----

```
def display_products(products, show_markup=False):
```

```
    """
```

Displays the list of products in a formatted table.

If `show\_markup` is True, the prices are shown with a 200% markup.

Parameters:

```
    -----
```

products : list

A list of dictionaries, each representing a product.

Required keys: 'sn', 'name', 'brand', 'quantity', 'price', 'country'.

show_markup : bool, optional

If True, the product price is displayed with a 200% markup. (Default is False)

Returns:

None

Raises:

ValueError:

 If there is an error while reading product data (e.g., non-integer price or quantity).

Header display

```
print("_ " * 80)
```

```
print("| S.N | Product Name      | Brand Name      | Qty | Price  | Country  |")
```

```
print("-" * 80)
```

```
i = 0
```

```
while i < len(products):
```

```
    p = products[i]
```

```
    # Extracting values
```

```
    name = p["name"]
```

```
    brand = p["brand"]
```

```
    try:
```

```
        qty = str(p["quantity"])
```

```
        sn = str(p["sn"])
```

```
        country = p["country"]
```

```
        price_val = int(p["price"]) * 3 if show_markup else int(p["price"])
```

```
    except ValueError:
```

```
        print("Error reading product data. Skipping item.")
```

```
        i += 1
```

```
        continue
```

```
# Formatting fields to fixed length
```

```
price = "NRP " + str(price_val)
```

```
# Manual alignment with spaces
```

```
while len(name) < 18:
```

```
    name = name + " "
```

```
while len(brand) < 18:
```

```
    brand = brand + " "
```

```
while len(qty) < 5:
```

```
    qty = qty + " "
```

```
while len(price) < 9:
```

```
    price = price + " "
```

```
while len(country) < 12:
```

```
    country = country + " "
```

```
if len(sn) == 1:
```

```

        sn = " " + sn

    # Displaying the product line
    print("| " + sn + " | " + name + " | " + brand + " | " + qty + " | " + price + " | " +
country + " |")
    i = i + 1

print("-" * 80)

```

Operation.py

```

import read
import write
import datetime
import random

# ----- Buy Product -----

def buy_product():
    """
    Handles the buying (restocking) of products, including updating the inventory
    and generating an invoice for the transaction.
    """
    products = read.read_products()
    if not products:
        return
    read.display_products(products)
    vendor_name = input("Enter vendor name: ")
    selected = []
    total = 0
    successful_transaction = False

    while True:
        sn = input("Enter S.N of product to restock (or 'new' to add new product, or 'done'
to finish): ")

        if sn.lower() == "done":
            break
        elif sn.lower() == "new":
            name = input("Enter product name: ")
            brand = input("Enter brand: ")
            try:
                quantity = int(input("Enter quantity to add: "))
                price = int(input("Enter cost price (before markup): "))
            except ValueError:

```

```

        print("Invalid input. Please enter valid numbers for quantity and price.")
        continue
    country = input("Enter country of origin: ")
    new_sn = str(len(products) + 1)

    new_product = {
        "sn": new_sn,
        "name": name,
        "brand": brand,
        "quantity": quantity,
        "price": price,
        "country": country
    }
    products.append(new_product)
    selected.append([name, brand, quantity, price, quantity * price])
    total += quantity * price
    print("New product " + name + " added with S.N " + new_sn + ".")
    successful_transaction = True

else:
    found = False
    for p in products:
        if p["sn"] == sn:
            found = True
            try:
                qty = int(input("Enter quantity to add: "))
                if qty <= 0:
                    print("Quantity must be a positive integer. Please try again")
                    break
            except ValueError:
                print("Invalid input.please enter a valid number")
                break
            try:
                new_price = int(input("Enter updated cost price(before markup)"))
            except ValueError:
                print("Invalid input.please enter a valid number")
                break

            p["quantity"] = int(p["quantity"]) + qty
            p["price"] = new_price
            amount = qty * new_price
            selected.append([p["name"], p["brand"], qty, new_price, amount])
            total += amount
            print("Product " + p["name"] + " restocked. New quantity: " +
str(p["quantity"]))
            successful_transaction = True

```

```

        break

write.write_products(products)

if successful_transaction:
    now = datetime.datetime.now()
    voucher = str(random.randint(1000, 9999))
    fname = "invoice_buy_" + voucher + "_" + now.strftime('%Y%m%d%H%M') + ".txt"
    invoice = "Voucher No: " + voucher + "\nVendor Name: " + vendor_name +
"\nDate: " + now.strftime('%Y-%m-%d %H:%M') + "\n\n"
    invoice += "Product      Brand      Qty  Price  Amount\n"

    for item in selected:
        name = item[0]
        brand = item[1]
        qty = str(item[2])
        price = str(item[3])
        amount = str(item[4])

        line = name + " " * (14 - len(name))
        line += brand + " " * (14 - len(brand))
        line += qty + " " * (6 - len(qty))
        line += price + " " * (8 - len(price))
        line += amount + "\n"
        invoice += line

    invoice += "\nTotal Amount: " + str(total)
    print(invoice)
    write.save_invoice(invoice, fname)
    print("Voucher Number (Buy): " + voucher)

# ----- Sell Product -----
def sell_product():
    """
    Handles the selling of products, including updating the inventory
    and generating an invoice for the transaction.

    This function allows the user to:
    - View the current list of products with their markup prices.
    - Sell products by entering their S.N (Serial Number).
    - Apply the "Buy 3, Get 1 Free" offer if applicable.
    - Generate an invoice for the transaction.

    Parameters:
    None

```

Returns:

None

Raises:

ValueError:

If the user enters invalid data types for quantity or incorrect phone number format.

.....

```
products = read.read_products()
```

```
if not products:
```

```
    return
```

```
read.display_products(products, show_markup=True)
```

```
# Get valid customer name
```

```
while True:
```

```
    customer_name = input("Enter customer name: ")
```

```
    if customer_name != "" and customer_name.isalpha():
```

```
        break
```

```
    else:
```

```
        print("Invalid customer name. Please enter a valid name (alphabets only).")
```

```
# Get valid customer phone number
```

```
while True:
```

```
    customer_phone = input("Enter customer phone number (10 digits): ")
```

```
    if customer_phone.isnumeric() and len(customer_phone) == 10:
```

```
        break
```

```
    else:
```

```
        print("Invalid phone number. Please enter a valid 10-digit phone number.")
```

```
selected = []
```

```
total = 0
```

```
successful_transaction = False
```

```
item_count = {}
```

```
while True:
```

```
    sn = input("Enter S.N of product to sell (or 'done'): ")
```

```
    if len(sn) == 4 and (sn[0] == 'd' or sn[0] == 'D') and (sn[1] == 'o' or sn[1] == 'O') and (sn[2] == 'n' or sn[2] == 'N') and (sn[3] == 'e' or sn[3] == 'E'):
```

```
        break
```

```
found = False
```

```
for p in products:
```

```
    if p["sn"] == sn:
```

```
        found = True
```

```
    try:
```

```
        qty = int(input("Enter quantity to sell: "))
```



```

except:
    print("Invalid quantity.")
    break

if qty <= 0:
    print("Quantity must be in positive positive number and greater than 0.")
    break

available = int(p["quantity"])
if sn not in item_count:
    item_count[sn] = 0
item_count[sn] += qty

# Apply "Buy 3, Get 1 Free" logic
free = item_count[sn] // 3
total_items = qty + free

if total_items > available:
    print("Not enough stock. Only " + str(available) + " items available.")
    break

unit_price = int(p["price"]) * 3
amount = qty * unit_price
p["quantity"] = available - total_items
selected.append([p["name"], p["brand"], qty, free, unit_price, amount])
total += amount
successful_transaction = True
print("Product " + p["name"] + " sold successfully with " + str(free) + " free
items.")
break

if not found:
    print("S.N not found.")

# Write to the file if the transaction was successful
if successful_transaction:
    write.write_products(products)
    print("\nTransaction Completed Successfully!\n")

# Generate Invoice
now = datetime.datetime.now()
voucher = str(random.randint(1000, 9999))
fname = "invoice_sell_" + voucher + "_" + now.strftime('%Y%m%d%H%M') + ".txt"

# Header Information
invoice = "Voucher No: " + voucher + "\n"

```

```

invoice += "Customer Name: " + customer_name + "\n"
invoice += "Phone Number: " + customer_phone + "\n"
invoice += "Date: " + now.strftime('%Y-%m-%d %H:%M:%S') + "\n\n"
invoice += "Product      Brand      Qty  Free  Price  Amount\n"

# Invoice Item Lines
for item in selected:
    name = item[0]
    brand = item[1]
    qty = str(item[2])
    free = str(item[3])
    price = str(item[4])
    amount = str(item[5])

    # Manual alignment with spaces
    line = name + " " * (14 - len(name))
    line += brand + " " * (14 - len(brand))
    line += qty + " " * (6 - len(qty))
    line += free + " " * (6 - len(free))
    line += price + " " * (8 - len(price))
    line += amount + "\n"
    invoice += line

# Total Amount
invoice += "\nTotal Amount: " + str(total)
print(invoice)

# Save the invoice
write.save_invoice(invoice, fname)
print("Voucher Number (Sell): " + voucher)

```

main.py

```

import operation
import read
import write
import datetime
import random

# ----- Main Program -----

def main():
    """
    Main function to run the WeCare Wholesale System.
    """

```

This function runs the main menu interface for the WeCare Wholesale System, allowing users to:

- View available products with their markups.
- Buy (restock) products.
- Sell products.
- Exit the system.

The system continues to loop through the menu options until the user selects 'Exit'.

Parameters:

None

Returns:

None

Raises:

Exception

If there is an unexpected error during program execution, it is caught and displayed to the user.

"""

while True:

try:

```
print("\nWELCOME TO WECARE WHOLESALE SYSTEM")
print("-----")
print("\nAddress: Bagbazar, Kathmandu | Ph. 9821000760")
print("-----")
```

```
# Display the list of products with their markup
read.display_products(read.read_products(), show_markup=True)
```

```
print("\nMenu:")
print("1. Buy Product (Restock)")
print("2. Sell Product")
print("3. Exit")
```

```
# User selects an option
choice = input("Enter Your choice (1/2/3): ")
```

```
if choice == "1":
    # Buy (Restock) product
    operation.buy_product()
elif choice == "2":
    # Sell product
```

```
        operation.sell_product()
    elif choice == "3":
        # Exit the application
        print("Thank you for using WeCare Wholesale System")
        break
    else:
        # If user enters an invalid option
        print("Invalid choice.")
except Exception as err:
    print("An unexpected error occurred:", err)

# Entry point of the program
if __name__ == "__main__":
    main()
```